



INTERNSHIP REPORT

OF

Python-Based Network Packet Sniffer

Submitted To

School of Computer Science and Engineering

IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement of degree of

B.Tech CSE

By

Riha Singh

CS-2341156

Batch: 2026-27

Internship Project

Network Packet Sniffing and Protocol-Based Traffic Analysis using Python and Scapy

1. CANDIDATE'S DECLARATION

I Riha Singh, do hereby declare that the internship report titled “Python-Based Network Packet Sniffer” has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in CSE. The work presented in this report is based on the internship project developed by me using Python, Scapy and system networking utilities. All references used in preparing this report have been appropriately acknowledged.

I affirm that this report is my own work and has not been submitted to any other institute for any degree or diploma requirement. I shall be solely responsible for any copyright violation in this regard.

Student Name: Riha Singh

Roll No.: CS-2341156

2. ACKNOWLEDGEMENT

I take this opportunity to express my sincere gratitude to everyone who supported and guided me throughout my internship. I am thankful for the valuable guidance, constructive criticism, technical suggestions and encouragement that helped me understand practical aspects of computer networking and network security.

I am especially grateful to my internship mentors and faculty members for providing an environment in which I could explore packet capture, network interfaces, protocol filtering and Python-based network analysis. Their guidance helped me connect theoretical concepts studied in the B.Tech CSE curriculum with practical implementation.

I also express my gratitude to my mentors and for their continuous encouragement and support throughout the internship.

Student Name: Riha Singh

Roll No.: CS-2341156

3. INTERNSHIP COMPLETION CERTIFICATE

A copy of the Internship Completion Certificate issued by the industry/organisation:



TABLE OF CONTENTS

Chapter No.	Chapter Name	Page Nos.
1.	Candidate's Declaration	2
2.	Acknowledgment	3
3.	Internship Completion Certificate	4
4.	Project Description	6
4.1	Introduction	6
4.2	Organization Profile	6
4.3	Problem Statement	6
4.4	Project Objectives	7
4.5	Scope of the Project	7
4.6	Technologies and Tools Used	8
4.7	System Architecture	8
4.8	Methodology	9
4.9	Expected Outcomes	10
4.10	Certificates of completion and communication proof	11
5.	Bibliography/References	14

4. PROJECT DESCRIPTION

4.1 Introduction

The internship project is a Python-based network packet sniffer designed to capture and inspect network packets from a user-selected network interface. The application uses Scapy's sniffing functionality to collect packets and display their detailed contents. It also uses the Linux `ip link` command to discover network interfaces and validate the interface selected by the user.

The project provides an interactive command-line workflow. At the beginning of each capture session, the program displays the available interfaces. The user selects an interface, specifies the number of packets to capture, and optionally provides a protocol filter. Captured packets are passed to a display function that uses Scapy's packet inspection capability to present packet-layer information.

The project demonstrates practical concepts of computer networking, packet capture, protocol analysis, operating-system network interfaces and Python automation. It can serve as a foundational component for more advanced network-monitoring and security-analysis systems.

4.2 Organization Profile

Organization Name: Code Alpha

Department / Team: CybersecurityIntern

Internship Duration: 20/11/2025 to 20/12/2025

Mentor / Supervisor: CodeAlpha team members

4.3 Problem Statement

Network communication generates a large volume of packets containing information associated with different protocols, endpoints and application activities. Manually observing this traffic is difficult and inefficient, especially when a learner or analyst needs to focus on a particular interface or protocol.

The problem addressed by this project is to provide a simple interactive utility that allows a user to select an available network interface, capture a specified number of packets or continue until manually stopped, optionally filter traffic using a protocol

expression, and inspect the captured packet structure. The implementation should also validate the selected interface before starting capture so that invalid interface names are handled gracefully.

4.4 Project Objectives

- To develop a command-line network packet-sniffing utility using Python and Scapy.
- To discover and display network interfaces available on the host system.
- To allow the user to select a specific interface for packet capture.
- To validate the selected interface before initiating sniffing.
- To support a user-defined packet capture limit, including unlimited capture when the limit is zero.
- To provide optional protocol-based filtering through a user-supplied filter expression.
- To display detailed information about each captured packet using Scapy.
- To strengthen practical understanding of packet capture, network interfaces and protocol analysis.

4.5 Scope of the Project

The current scope covers local packet capture from a user-selected network interface, interactive capture configuration, basic protocol filtering and packet inspection. The project is primarily intended for educational and authorized network-analysis environments.

The implementation does not currently include persistent packet storage, a graphical dashboard, automated threat classification, packet statistics, alert generation, packet reconstruction, or long-term reporting. These capabilities can be considered future enhancements.

Because packet sniffing can expose network information, the utility should only be used on systems and networks where the user has appropriate authorization.

4.6 Technologies and Tools Used

Technology / Tool	Role in Project	Application
-------------------	-----------------	-------------

Python	Programming language	Application logic, input handling, validation and control flow
Scapy	Packet manipulation and capture library	Packet sniffing and packet-layer inspection
subprocess	Python standard library module	Execution of the Linux ip link command
Linux ip link	Network interface utility	Listing and validating available interfaces
Command Line Interface	User interaction	Selecting interface, capture limit and protocol filter

4.7 System Architecture

The functional architecture of the application can be represented as the following sequence:

User → Interface Discovery → Interface Selection → Interface Validation → Capture Configuration → Scapy Sniffer → Packet Display

Interface Discovery: The `display_interfaces()` function invokes the Linux ip link command through `subprocess.run()` and prints the command output. It also extracts interface names from the returned text.

Interface Selection: The user is prompted to enter the interface on which packet capture should occur.

Interface Validation: The `is_valid_interface()` function executes ip link again, extracts interface names and checks whether the entered interface exists.

Capture Configuration: The user specifies a packet count. A value of zero is interpreted as no packet-count limit. The user can also enable or disable protocol filtering and, when enabled, enter a filter expression.

Packet Capture: Scapy's `sniff()` function receives the selected interface, packet callback, count and optional filter. Each captured packet is passed to `show_packet()`.

Packet Inspection: `show_packet()` calls `packet.show()`, which displays the packet's available protocol-layer fields and values.

Session Control: After a capture session ends, the user is asked whether the application should exit. If the response is not Y, the workflow starts another capture session.

4.8 Methodology

4.8.1 Interface Discovery

The program first calls `display_interfaces()`. Inside this function, `subprocess.run(['ip', 'link'], capture_output=True, text=True, check=True)` executes the system command and captures its output. The program then parses lines containing a colon to construct a list of interface names. This provides the user with a view of interfaces available on the system.

4.8.2 Interface Validation

After the user selects an interface, `is_valid_interface()` executes `ip link` and creates the interface list again. The expression `return interface in interfaces` verifies whether the selected name exists. If validation fails, an error is displayed and the program returns to the selection stage.

4.8.3 Capture Configuration

The user enters the number of packets to capture. The expression `limit if limit else 0` converts zero into the unlimited-capture value expected by the Scapy sniffing workflow. The program also asks whether protocol filtering is required. When enabled, the supplied filter expression is passed to Scapy.

4.8.4 Packet Sniffing

The program builds a parameter dictionary containing `iface`, `prn`, `count` and `filter`. The `iface` value identifies the capture interface, `prn` points to `show_packet` as the packet callback, `count` controls the number of packets and `filter` contains the optional capture filter. The dictionary comprehension removes empty values before `sniff()` is invoked.

4.8.5 Packet Display

The `show_packet(packet)` callback receives each captured packet and calls `packet.show()`. Scapy consequently displays the packet structure and fields available in the captured protocol layers. This allows the user to examine information such as protocol layers and field values without implementing a packet parser from scratch.

4.8.6 Repeated Capture Sessions

The complete process is enclosed in a while True loop. After each capture session, the program asks the user whether to exit. Entering Y breaks the loop; otherwise the program starts another session and again displays available interfaces.

4.8.7 Core Implementation Logic

The project is organized into four main functions: sniff_packets(), show_packet(), display_interfaces(), and is_valid_interface(). sniff_packets() controls the overall user workflow; show_packet() handles packet presentation; display_interfaces() discovers interfaces; and is_valid_interface() checks the user's selection. The if __name__ == '__main__' block starts the application by calling sniff_packets().

4.8.8 Operational Requirements and Safety

Packet capture may require appropriate operating-system privileges depending on the environment and interface configuration. The project should therefore be executed in an authorized lab, internship environment or network owned/managed by the user. The tool should not be used to intercept traffic without permission.

4.9 Expected Outcomes

- A functional command-line packet-sniffing utility capable of capturing traffic from a selected interface.
- Successful discovery and validation of available Linux network interfaces.
- User-controlled packet-count configuration, including unlimited capture when zero is selected.
- Optional protocol filtering through a Scapy-compatible filter expression.
- Detailed packet-layer output for captured traffic.
- Improved practical understanding of Python networking, Scapy, packet capture and Linux network interfaces.
- A foundation for future development of network-monitoring and security-analysis capabilities.

4.10 Certificates of Completion and Other Communication Mail Proof

Internship Completion Certificate



Internship Offer / Joining Confirmation



YOUR DREAM OUR PASSION

INTERNSHIP OFFER LETTER

15th November 2025

Name: Riha Singh

STUDENT ID: CA/SE1/7531

Welcome to CodeAlpha

Congratulations! We are delighted to make you an offer as **Cyber Security Internship** offer letter is to confirm your selection **Cyber Security Internship** at **CodeAlpha** and welcome you for the same effective from 20th November 2025 to 20th December 2025. Hope you will be doing well.

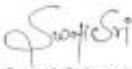
This Internship is observed by CodeAlpha as being a learning opportunity for you.

Your internship will embrace orientation and give emphasis on learning new skills with a deeper understanding of concepts through hands-on application of the knowledge you gained as an intern. You will acknowledge your obligation to perform all work allocated to you to the best of your ability within lawful and reasonable direction given to you.

We look forward to a worthwhile and fruitful association which will make you equipped for future projects.

Wishing you the most enjoyable and truly meaningful internship program experience.

Sincerely,



Swati Srivastava
Founder & CEO







Phone
+91 9036576683



Email
services@codealpha.tech



Address
Lucknow U.P

Letter Of Recommendation:

 Website: www.codealpha.tech Email ID: services@codealpha.tech support@codealpha.tech LinkedIn: www.linkedin.com/company/codealpha Mobile No: +91 9336576683	CODEALPHA YOUR DREAM OUR PASSION Letter Of Recommendation TO WHOM IT MAY CONCERN This is to certify that Riha Singh has successfully completed the Virtual Internship Program at CodeAlpha for a duration of one month from 20th November 2025 to 20th December 2025 as a Cyber Security Internship. He/She exhibited performance in this role and made a valuable contribution to our organization during the internship period. He/She has excellent analytical skills and He/She quickly acquired new skills and adeptly adapted to emerging technologies, demonstrating a high level of productivity. He/She consistently displayed a willingness to provide assistance and fostered strong collaboration with fellow team members. His/Her dedication and skills make him/her a valuable asset to any prospective employer, and CodeAlpha wholeheartedly recommend him/her for any future endeavor. REGARDS,  FOUNDER & CEO  21st December 2025 DATE

5. BIBLIOGRAPHY / REFERENCES

1. **Scapy Documentation**, “Scapy: Packet Manipulation and Network Security Tool,” Scapy Project. Available at: <https://scapy.readthedocs.io/>
2. **Python Software Foundation**, “Python 3 Documentation,” Python Documentation. Available at: <https://docs.python.org/3/>
3. **Python Software Foundation**, “subprocess — Subprocess Management,” Python Standard Library Documentation. Available at: <https://docs.python.org/3/library/subprocess.html>
4. **Linux Manual Pages**, “ip-link — network device configuration,” Linux Networking Documentation. Available at: <https://man7.org/linux/man-pages/man8/ip-link.8.html>
5. **Wireshark Foundation**, “Wireshark User’s Guide,” Wireshark Documentation. Available at: <https://www.wireshark.org/docs/>
6. **NIST**, “Guide to Intrusion Detection and Prevention Systems (IDPS),” NIST Special Publication 800-94, National Institute of Standards and Technology.